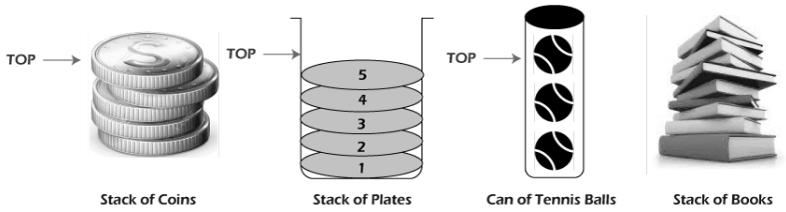
Les piles

Le concept de pile est basé sur celui d'une pile d'assiettes ou de crêpes : on ajoute des assiettes ou des crêpes sur la pile, et on les récupère dans l'ordre inverse, en commençant par la dernière ajoutée.



Définition

En informatique, une pile (*stack*) est une structure de données (un type de variable) pour lesquelles on dispose des méthodes suivantes :

- `empile(elem)` : .....
- `depile()` : .....

Selon les langages, on dispose en général des méthodes supplémentaires suivantes :

- `est_vide()` : .....
- `taille()` : .....

Le langage Python ne propose pas de type « pile » au sens strict du terme. En revanche, le type `list` de Python est une bonne base pour créer une structure de données de pile. C'est ce qui est proposé dans le fichier pile.py, à télécharger pour réaliser les exercices suivants.

/!\ Pour répondre aux exercices, et il est interdit d'utiliser le type `list` de Python.

/!\ Il est interdit de mofifier la classe `Pile`, à moins que ce soit explicitement demandé.

Exercice 1 – *(+)*

On considère le script ci-dessous.

```
p = Pile()
p.empile("n")
p.empile("s")
p.empile("i")
x = p.depile()
p.depile()
p.empile(x)
```

1. Donner l'état de la pile `p` à l'issue de ce script.

2. Parmi les modes FIFO (First In First Out) et LIFO (Last In First Out), lequel s'applique aux piles ?

Exercice 2 – *(+)*

Écrire une fonction `echange(p)` qui permute les deux éléments situés en haut de la pile `p` (on part du principe qu'elle contient au moins deux éléments). Faire les affichages permettant de montrer que le programme effectue ce qui est attendu. La modification de la pile a lieu « en place », c'est-à-dire qu'on n'a pas à renvoyer la pile car elle sera modifiée à l'intérieur et à l'extérieur de la fonction demandée.

Exercice 3 – *(++)*

Écrire une fonction `inverse(p)` qui renvoie une nouvelle pile constituée des mêmes éléments que `p`, mais dans l'ordre inverse.

Attention : la pile passée en paramètre doit être inchangée à la sortie de la fonction !

Indice : utiliser une pile auxiliaire.

Exercice 4 – *(++)*

Écrire une fonction `copie(p)` qui renvoie une nouvelle pile dont les éléments sont ceux de `p`, dans le même ordre.

Exercice 5 – *(++)*

1. Écrire une fonction `fond(p)` qui supprime et renvoie l'élément situé en bas de la pile `p`.
2. Écrire une fonction `rotation(p)` qui échange l'élément situé en haut de la pile avec celui situé en bas.

Exercice 6 – *(+++)*

Une expression est correctement parenthésée si à chaque signe de parenthésage ouvert correspond un signe de parenthésage fermé de même forme, et s'il n'y pas de « croisement » entre les correspondances.

Écrire une fonction `test_parenthesage` qui prend en argument une chaîne de caractères et renvoie `True` si celle-ci est correctement parenthésée, et `False` sinon.

On donne ci-dessous un jeu de tests pour détecter d'éventuels défauts de conception.

```
assert test_parenthesage("(a)(b)(((c)(d)))")==True
assert test_parenthesage("([b]){((c))[d]}")==True
assert test_parenthesage("(")==False
assert test_parenthesage("(a))")==False
assert test_parenthesage("((a))}")==False
assert test_parenthesage("[[a]")==False
assert test_parenthesage("[(a])")==False
assert test_parenthesage("{((a)})")==False
```

Exercice 7 – *(+++)*

On considère une pile `p` d'assiettes de couleur blanche ou verte, chacune étant identifiée par un numéro. On modélise une assiette par un tuple `(couleur, numero)`.

Concevoir une fonction `ranger` qui range une pile d'assiettes en plaçant les assiettes vertes au dessus des blanches, mais sans changer l'ordre relatif des numéros.